`MixedComplementarityProblems.jl`: A Fast, Batched, Open-Source Interior-Point Solver for Mixed Complementarity Problems

David Fridovich-Keil
Department of Aerospace Engineering and Engineering Mechanics
The University of Texas at Austin
Austin, TX, USA
dfk@utexas.edu

***Abstract*—Mixed complementarity problems (MCPs) arise as the first-order optimality conditions of nonlinear programs and noncooperative games, and give a natural formulation for the multi-agent trajectory optimization problems that appear throughout robotics. The dominant solver for problems of this form is `PATH`, a mature but closed-source package. We present `MixedComplementarityProblems.jl`, an open-source, pure Julia implementation of an interior-point method for parametric MCPs that (i) matches `PATH`'s reliability on standard benchmarks and (ii) natively supports batched solving, in which many parameter instances are solved simultaneously and in parallel, either across CPU threads or on an NVIDIA GPU, behind a single solver implementation. On a multi-agent lane-change trajectory game representative of robotics planning problems, our CPU-multithreaded batched solver clears a batch of parametric instances 60–80$\times$ faster than sequential calls to `PATH`, and our GPU backend clears the same batches roughly $20\times$ faster than `PATH` [TODO: finalize the GPU-vs-CPU comparison for the trajectory game once the latest benchmark numbers are in—earlier runs showed GPU trailing CPU here, but that may no longer hold]. We describe the solver's interior-point formulation, the abstraction that lets a single solver implementation run unmodified across dense, batched-sparse, and single-large linear-algebra backends, and report benchmarks against `PATH` on both randomly generated quadratic programs and trajectory games.**

I. Introduction

Multi-agent trajectory planning problems in robotics, ranging from autonomous driving to human-robot interaction, are naturally posed as noncooperative dynamic games, in which each agent optimizes its own trajectory subject to the others' choices. The first-order necessary conditions of optimality for each agent in the game take the form of a mixed complementarity problem (MCP). Solving MCPs rapidly, therefore, becomes a core computational burden for real-time decision-making in multi-agent robotics applications.

The dominant solver for this class of problems is `PATH` [1]. `PATH` is mature, reliable, and widely used; but, it is closed-source, solves one problem instance at a time, and was not designed to support automatic differentiation of solutions with respect to problem parameters. All three pose serious, practical limitations for use in robotics: (i) developers must have access to solver internals in order to customize to the problem at hand, (ii) scenario- and contingency-based planning (e.g., [2], [3]) requires solving a *batch* of identically-structured but differently *parameterized* problems, and (iii) modern machine learning pipelines often embed optimization problem solvers as "layers" [4], and end-to-end differentiability requires us to be able to differentiate a solver's output with respect to its input parameters.

We present `MixedComplementarityProblems.jl`, an open-source, pure Julia solver for parametric MCPs that addresses all of these practical challenges. Concretely, we contribute:

- A simple and easily-customized interior-point method for parametric MCPs that matches `PATH`'s reliability on standard benchmarks, while supporting automatic differentiation of solutions with respect to problem parameters.
- A native *batched* solve mode, in which many parameter instances sharing one MCP structure are solved simultaneously, parallelized across CPU threads or an NVIDIA GPU behind a single solver implementation.
- An abstraction boundary defined by a small verb set that separates the interior-point loop and its linear algebra backend, which lets a single solver implementation run unmodified across dense, batched-sparse, and single-large-instance regimes, so that supporting a new device or linear-algebra strategy never requires touching the solver loop itself.

As Figure 1 previews and Section V quantifies, on a multi-agent lane-change trajectory game representative of robotics planning problems, clearing a batch of parametric instances with our batched solver takes a small fraction of the wall-clock time required to clear the same batch with sequential calls to `PATH`. **[TODO: replace with the top-line numerical speedup results when they are finalized]**

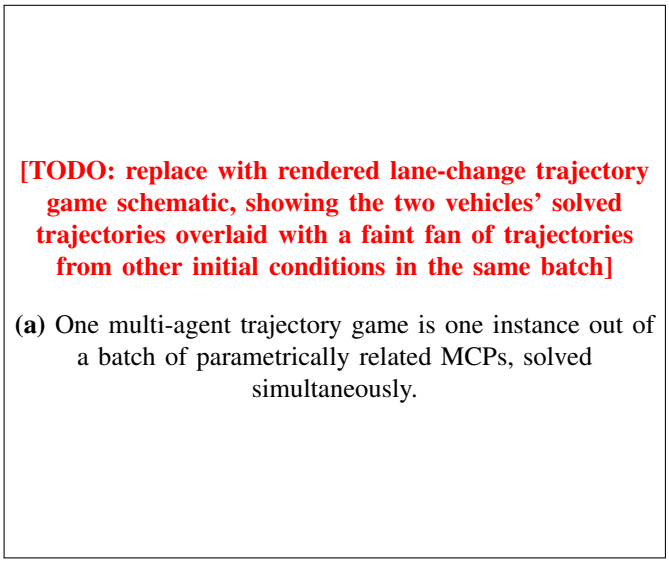
II. Background: Mixed Complementarity Problems

A. Problem definition

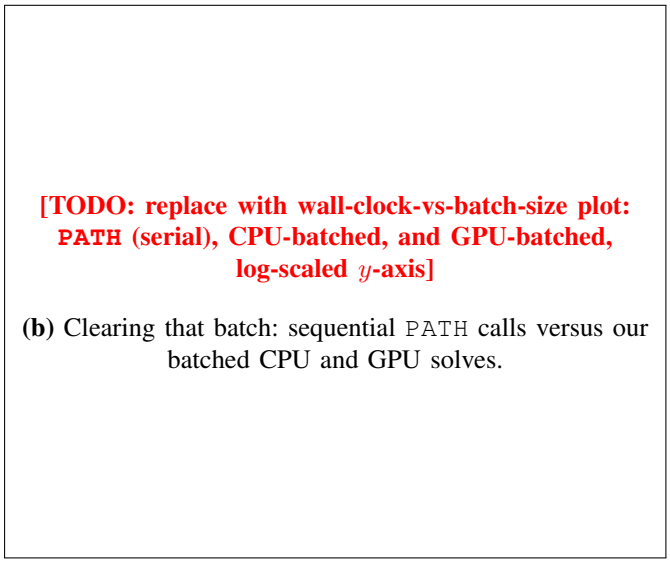
A mixed complementarity problem is specified by a map $F : \mathbb{R}^n \to \mathbb{R}^n$ and bounds $\ell, u \in (\mathbb{R} \cup \{\pm\infty\})^n$; a solution is a vector $z \in \mathbb{R}^n$ satisfying, for every coordinate dimension i,

$$\begin{aligned} \ell_i < z_i < u_i &\implies F_i(z) = 0, \\ z_i = \ell_i &\implies F_i(z) \geq 0, \\ z_i = u_i &\implies F_i(z) \leq 0. \end{aligned} \tag{1}$$

Readers are referred to [5] for a complete introduction to MCPs and related problems.



(a) One multi-agent trajectory game is one instance out of a batch of parametrically related MCPs, solved simultaneously.



(b) Clearing that batch: sequential `PATH` calls versus our batched CPU and GPU solves.

Fig. 1: **[TODO: finalize caption once the real figure is in]** Solving a batch of parametric trajectory-game instances (a) with `MixedComplementarityProblems.jl`'s batched interior-point solver is substantially faster than solving the same batch with sequential calls to `PATH` (b).

B. MCPs from KKT conditions

Consider a parametric nonlinear program

$$\min_{x} \; f(x;\theta) \quad \text{s.t.} \quad g(x;\theta) = 0,\; h(x;\theta) \geq 0, \tag{2}$$

with primal variables $x \in \mathbb{R}^{n_x}$, parameters θ, and Lagrange multipliers $\lambda \in \mathbb{R}^{n_\lambda}$ associated with the equality constraint g and $\mu \in \mathbb{R}^{n_\mu}_{\geq 0}$ associated with the inequality constraint h. The first order necessary (i.e. Karush-Kuhn-Tucker, KKT) conditions of optimality for problem (2) are

$$0 = \overbrace{\begin{bmatrix} \nabla_x f(x;\theta) - \nabla_x g(x;\theta)^\top \lambda - \nabla_x h(x;\theta)^\top \mu \\ g(x;\theta) \end{bmatrix}}^{G(x,\lambda,\mu;\theta)} \tag{3a}$$

$$0 \leq \underbrace{h(x;\theta)}_{H(x;\theta)} \perp \mu \geq 0. \tag{3b}$$

Problem (3) can be converted into the form (1) by taking $z = (x, \lambda, \mu)$, $F = (G, H)$, $u_i = \infty$ $(\forall i)$, $\ell_i = -\infty$ $(\forall i \in \{1, 2, \ldots, n_x + n_\lambda\})$, and $\ell_i = 0$ $(\forall i > n_x + n_\lambda)$. Because of the natural connection between KKT conditions (e.g., arising from trajectory optimization problems in robotics) and this *primal-dual* form of the MCP, it is the only form `MixedComplementarityProblems.jl` exposes.

C. MCPs from noncooperative games

The construction in Section II-B extends from one decision maker to many. Consider N agents, where agent i solves

$$\forall i \begin{cases} \min_{x_i} \; f_i(x_i, x_{\neg i};\theta) \\ \text{s.t.} \quad g_i(x_i, x_{\neg i};\theta) = 0,\; h_i(x_i, x_{\neg i};\theta) \geq 0 \end{cases} \tag{4a}$$

$$\text{s.t.} \quad g_{\text{sh}}(x;\theta) = 0,\; h_{\text{sh}}(x;\theta) \geq 0, \tag{4b}$$

subject to its own equality and inequality constraints (g_i and h_i, respectively) and to constraints $g_{\text{sh}}, h_{\text{sh}}$ shared across agents (e.g., pairwise collision avoidance). Each agent i's decision variable is $x_i \in \mathbb{R}^{n_{x_i}}$, and $x_{\neg i}$ denotes the other agents' decisions so that $x = (x_i, x_{\neg i}) = (x_1, \ldots, x_N)$. Writing out each agent's KKT conditions as in (3) and concatenating them—with the shared constraints $g_{\text{sh}}, h_{\text{sh}}$ coupling agents through shared multipliers $\lambda_{\text{sh}}, \mu_{\text{sh}}$—yields a single primal-dual MCP whose solution is a generalized Nash equilibrium: a strategy profile from which no agent can unilaterally improve its own objective [5].

Concretely, concatenate the Lagrange multipliers corresponding to equality constraints as $\lambda := (\lambda_1, \ldots, \lambda_N, \lambda_{\text{sh}})$, and those for inequality constraints as $\mu := (\mu_1, \ldots, \mu_N, \mu_{\text{sh}})$. Then, private multipliers as $\lambda_{\text{pr}} = (\lambda_1, \ldots, \lambda_N)$, $\mu_{\text{priv}} = (\mu_1, \ldots, \mu_N)$. Each agent i's Lagrangian is given by

$$\begin{aligned} \mathcal{L}_i(x, \lambda, \mu;\theta) := f_i(x;\theta) - \lambda_i^\top g_i(x;\theta) - \mu_i^\top h_i(x;\theta) - \\ \lambda_{\text{sh}}^\top g_{\text{sh}}(x;\theta) - \mu_{\text{sh}}^\top h_{\text{sh}}(x;\theta). \end{aligned} \tag{5}$$

The N agents' concatenated KKT conditions are then an instance of (3), with

$$\begin{aligned} G &:= (\nabla_{x_1} \mathcal{L}_1, \ldots, \nabla_{x_N} \mathcal{L}_N, g_1, \ldots, g_N, g_{\text{sh}}), \\ H &:= (h_1, \ldots, h_N, h_{\text{sh}}), \end{aligned} \tag{6}$$

with G and H functions of (x, λ, μ) and parameterized by θ.

Running example: A lane-change trajectory game. Consider the two-agent lane changing trajectory game of Figure 1(a). Here, each agent's decision variable $x_i = \{s_i(t), a_i(t)\}_{t=1}^T$ consists of a state-action *trajectory* over time horizon T. The private equality constraints g_i encode system dynamics $\big(s_i(t), a_i(t)\big) \to s_i(t+1)$, private inequality constraints encode actuator limits $\|a_i(t)\| \leq \bar{a}_i$, and shared inequality constraints encode collision-

avoidance $\|P(s_1(t) - s_2(t))\| \geq \Delta$ (with projection matrix P recovering agents' position). Agents' objective functions f_i encode lane and speed preferences, and penalize control effort. Parameters θ encode the agents' initial states $s_i(1)$, lane preferences, desired speeds, actuator limits $\bar{a}_i$, and collision-avoidance radius Δ.

Observe that the shared collision avoidance constraint—and any nonlinearity in agents' dynamics—renders the feasible set nonconvex.

III. Solver Design

`MixedComplementarityProblems.jl` implements an intentionally-simple primal-dual interior point method to solve problem (3). To that end, we introduce a slack variable $z \in \mathbb{R}^{n_\mu}$ and rewrite (3) in terms of $z \geq 0$ and a homotopy parameter $\rho > 0$:

$$K_\rho(x, \lambda, \mu, z; \theta) := \begin{bmatrix} G(x, \lambda, \mu; \theta) \\ H(x; \theta) - z \\ z \odot \mu - \rho 1 \end{bmatrix} = 0. \tag{7}$$

Note that the last row of K_ρ involves the elementwise product $z \odot \mu$ and, consequently, the number of equations in K_ρ is equal to the total dimension of the primal, dual, and slack variables (x, λ, μ, z).

As summarized in Algorithm 1, we proceed in rounds by solving (7) via a regularized Newton method and decaying $\rho \to 0$ until reaching the user's defined tolerance $\|K_0\| \leq \bar{K}$. For a given value of ρ, the regularized Newton direction $\delta := (\delta_x, \delta_\lambda, \delta_\mu, \delta_z)$ solves

$$\left(\begin{bmatrix} \nabla_x G & \nabla_\lambda G & \nabla_\mu G & 0 \\ \nabla_x H & 0 & 0 & -I \\ 0 & 0 & \mathrm{dg}(z) & \mathrm{dg}(\mu) \end{bmatrix} + \epsilon I \right) \delta = -K_\rho, \tag{8}$$

with $\epsilon \geq 0$ a regularization parameter chosen as described below, and $\mathrm{dg}(\cdot)$ returning a square matrix with its argument on the main diagonal and all other entries zero.

We implement two additional subtleties. First, on line 5, we conduct a "fraction to the boundary" linesearch to control the rate at which $z, \mu \geq 0$ can approach 0 (cf. **[TODO: reference to Nocedal and Wright's book where they talk about this]**). At each iteration k, stepsizes α_k, β_k are chosen so that

$$\begin{aligned} \alpha_k &= \max\left(\alpha \in [0,1] \;:\; z + \alpha \delta_z \geq (1-\tau) z\right) \\ \beta_k &= \max\left(\beta \in [0,1] \;:\; \mu + \beta \delta_\mu \geq (1-\tau) \mu\right), \end{aligned} \tag{9}$$

where the inequalities are interpreted elementwise and $\tau \in (0,1)$ controls the rate of decay. Linesearch (9) has a closed-form, exact solution, which makes it amenable to batched computation as described in Section IV.

Second, we construct an *adaptive* choice of the Newton step regularization parameter ϵ and the interior point homotopy parameter ρ that aims to control subproblem conditioning. At the end of each inner loop (lines 10 and 12), if the solver successfully drove $\|K_\rho\| \leq \rho$ then we *tighten* ρ and ϵ at a rate dependent upon the number of Newton steps required to converge; conversely, if the inner loop could not meet the tolerance and $\|K_\rho\| > \rho$, we *loosen* both ρ and ϵ to improve the conditioning of the next subproblem and require a less precise solution.

Algorithm 1: Primal-dual interior-point method for (3)

Input: initial iterate (x, λ, μ, z) with $\mu, z > 0$;
parameters θ; initial $\rho, \epsilon > 0$;
tightening/loosening rates $\underline{\gamma}, \bar{\gamma} \in (0,1)$; max. inner iterations $k_{\max}$; tolerance $\bar{K}$

Output: (x, λ, μ, z) with $\|K_0(x, \lambda, \mu, z; \theta)\| \leq \bar{K}$

1 **while** $\|K_0(x, \lambda, \mu, z; \theta)\| > \bar{K}$ **do**
2 $k \leftarrow 0$
3 **repeat**
4 assemble K_ρ (7) and solve (8) for the Newton direction $\delta = (\delta_x, \delta_\lambda, \delta_\mu, \delta_z)$
5 compute step sizes α_k, β_k via the fraction-to-boundary rule (9)
6 $x \leftarrow x + \alpha_k \delta_x, \quad \lambda \leftarrow \lambda + \alpha_k \delta_\lambda,$ $z \leftarrow z + \alpha_k \delta_z, \quad \mu \leftarrow \mu + \beta_k \delta_\mu$
7 $k \leftarrow k + 1$
8 **until** $\|K_\rho(x, \lambda, \mu, z; \theta)\| \leq \rho$ *or* $k \geq k_{\max}$
9 **if** $\|K_\rho(x, \lambda, \mu, z; \theta)\| \leq \rho$ **then**
10 $\rho \leftarrow \rho\,(1 - e^{-\underline{\gamma} k}), \quad \epsilon \leftarrow \epsilon\,(1 - e^{-\underline{\gamma} k})$ `// tighten`
11 **else**
12 $\rho \leftarrow \rho\,(1 + e^{-\bar{\gamma} k}), \quad \epsilon \leftarrow \epsilon\,(1 + e^{-\bar{\gamma} k})$ `// loosen`
13 $\rho \leftarrow \min(\rho, 1)$
14 **return** (x, λ, μ, z)

IV. Batched and GPU-Parallel Solving

[TODO: notation cleanup before shipping: (i) subscripts on primal/dual variables mean "player i" in Section II-C but "batch instance b" here—pick distinct symbols, or add an explicit disambiguating remark; (ii) batched primal/dual iterates are capitalized $(X, \Lambda, M, Z, \Theta)$ but the batched stepsizes $\alpha_k, \beta_k \in \mathbb{R}^B$ are not—make the capitalization convention consistent across both]

Many applications require solving not one but a whole *batch* of MCPs that share the same symbolic structure (G, H) and differ only in their parameter vector θ—for example, if we wished to solve the trajectory game of Figure 1 from many initial conditions. Rather than naively deploying Algorithm 1 on each instance in series, we reuse the shared symbolic structure and exploit hardware-level parallelism (both CPU multithreading and GPU) to solve an entire batch of B parametrically-related instances in a single call.

Running example: A batch of lane-change games. Sampling B initial conditions, lane preferences, or collision radii of Figure 1(a) yields B instances of the same symbolic game, differing only in θ; stacking them column-wise into $\Theta \in \mathbb{R}^{n_\theta \times B}$ and calling Algorithm 2 solves the entire batch simultaneously, e.g., to evaluate a

planner across a distribution of scenarios [2] rather than one scenario at a time.

A. A device- and strategy-agnostic solver loop

For convenience, we stack the batch's parameter vectors column-wise into $\Theta \in \mathbb{R}^{n_\theta \times B}$, so that column b is instance b's parameter vector θ_b; likewise, we stack each instance's iterate into $X, \Lambda, M, Z \in \mathbb{R}^{(\cdot)\times B}$. Algorithm 1 is then implemented *once*, against a small set of verbs—`residual!`, `jacobian!`, `factorize!`, `ldiv!`—that consume and return plain $(\cdot) \times B$ arrays. The numeric representation of the batched Jacobian ∇K_ρ and its factorization never leave these four verbs: they are encapsulated inside a strategy-specific cache, so the solver's control flow is agnostic to both *where* it runs and *how* ∇K_ρ is represented and factored. We implement a "batched sparse" strategy: the B instances share one sparsity pattern for ∇K_ρ, computed once from the symbolic (G, H), and each instance fills only its own column of the corresponding $(\mathrm{nnz} \times B)$ value array containing only the structurally nonzero elements of ∇K_ρ.

B. Adapting Algorithm 1 to a batch

Three changes distinguish the batched loop in Algorithm 2 from Algorithm 1. First, the homotopy and regularization parameters ρ, ϵ, and the convergence check $\|K_\rho\|$, all become length-B vectors, one per instance, so that harder problem instances can iterate longer without penalizing easier instances in the same batch.

Second, because linesearch (9) has a closed-form solution, the batched stepsizes $\alpha_k, \beta_k \in \mathbb{R}^B$ are therefore computed elementwise. There is no backtracking loop and, in particular, no per-instance synchronization between host and device.

Third, a batch can often include easy, hard, and infeasible instances. Therefore, after an instance exceeds a maximum number (e.g., 5) of consecutive outer iterations without satisfying $\|K_\rho\| \le \rho$, it is flagged as a failure. Depending upon the hardware backend (CPU or GPU), these failures are either skipped or retained, but kept frozen. This automatic detection of failures prevents a handful of stalled instances from slowing down a batch composed of many easier instances.

Stalled instances are not left to run: line 6 restricts the (re)assembly, factorization, and solve to active instances. This prevents a handful of stalled or failed instances from consuming Newton iterations once flagged; otherwise, they would force the entire batch to consume the maximum iteration count.

C. Backends

On CPU, the shared sparsity pattern is assembled once and each instance's numeric factorization of ∇K_ρ reuses the pivot ordering computed on the first outer round; the B instances are distributed across threads. Because each instance's factorization is an independent per-instance sparse LU (KLU) **[TODO: ref]**, skipping an inactive instance at line 6 is a literal per-instance branch: an instance with $\text{status}_b \neq$ `active` costs nothing beyond a boolean check, so CPU throughput scales with the number of instances still active, not B.

Algorithm 2: Batched primal-dual interior-point

Input: batch of B instances $(x_b, \lambda_b, \mu_b, z_b; \theta_b)$ as in Algorithm 1, each with its own ρ_b, ϵ_b; tightening/loosening rates $\underline{\gamma}, \bar{\gamma}$; max. inner/outer iterations $k_{\max}, T_{\max}$; max. consecutive stalled outer rounds T_{stall}; tolerance $\bar{K}$

Output: $\text{status}_b \in \{\texttt{solved}, \texttt{failed}\}$ and $(x_b, \lambda_b, \mu_b, z_b)$ for $b = 1, \ldots, B$

1 $\text{status}_b \leftarrow$ `active`, $\text{stall}_b \leftarrow 0 \quad \forall b$
2 $t \leftarrow 0$
3 **while** $t < T_{\max}$ *and* $\text{status}_b =$ `active` *for some* b **do**
4 $k \leftarrow 0$
5 **repeat**
6 assemble and factor ∇K_{ρ_b} and solve for δ_b, for every b with $\text{status}_b =$ `active`
7 step active instances via fraction-to-boundary (9); instances with $\text{status}_b \neq$ `active` take a zero step
8 $k \leftarrow k + 1$
9 **until** $\|K_{\rho_b}\| \le \rho_b$ *for every active* b, *or* $k \ge k_{\max}$
10 **foreach** b *with* $\text{status}_b =$ `active` **do**
11 **if** $\|K_{\rho_b}\| \le \rho_b$ **then**
12 **if** $\rho_b \le \bar{K}$ **then**
13 $\text{status}_b \leftarrow$ `solved`
14 **else**
15 tighten ρ_b, ϵ_b (as in Algorithm 1); $\text{stall}_b \leftarrow 0$
16 **else**
17 loosen ρ_b, ϵ_b; $\text{stall}_b \leftarrow \text{stall}_b + 1$
18 **if** $\text{stall}_b \ge T_{\text{stall}}$ **then** $\text{status}_b \leftarrow$ `failed`
19 $t \leftarrow t + 1$
20 **return** $\{\text{status}_b, (x_b, \lambda_b, \mu_b, z_b)\}_{b=1}^B$

The GPU backend cannot make the same trade. It factors and solves the batch's shared sparsity pattern with `cuDSS`'s *batched* sparse solver, which processes all B instances as a single device-side call and there is no per-instance skip inside a batched factorization. Consequently, line 6's active-instance restriction is accepted on GPU only to maintain a uniform verb signature across backends, but is a no-op there: every outer round factors and solves *all* B instances, converged and failed ones included, and only their zero stepsize keeps the frozen instances' $(x_b, \lambda_b, \mu_b, z_b)$ unchanged. This imposes a real cost on GPU—one paid in exchange for offloading the whole batch to a single, highly parallel factorization—and it is the reason T_{stall} matters more on GPU than on CPU.

Remark 1. *The verb set of Section IV-A is deliberately narrow in order to readily accommodate other strategies tailored to different problem structures. For example, a "batched dense" strategy could represent ∇K_ρ densely, as a $(\cdot)\times(\cdot)\times B$ tensor, and solve it with a batched LU factorization.*

V. Experiments

We evaluate `MixedComplementarityProblems.jl` against `PATH` along the two axes that matter for the robotics applications of Section I: reliability—*does the solver converge as often as `PATH`?*—and throughput—*how quickly can it clear a batch of parametrically-related instances?* Concretely, our experiments answer four questions:

1) **Single-instance parity.** Solving one problem at a time, does our interior-point method match `PATH`'s reliability, and how does its per-solve wall-clock compare? (Section V-B)
2) **Batched throughput.** When a whole batch must be cleared, how much faster is a single batched solve than sequential `PATH` calls? (Section V-C)
3) **CPU thread scaling.** How does batched throughput scale with the number of CPU threads? (Section V-D)
4) **GPU vs. CPU.** When does offloading the batch to a GPU beat a many-threaded CPU, and why? (Section V-E)

A. Experimental setup

a) Problem families: We benchmark on two families of parametric MCPs. The first is a family of randomly generated sparse, convex *quadratic programs* (QPs) of the form $\min_x \frac{1}{2}x^\top Mx - \phi^\top x$ s.t. $Ax - b \geq 0$, with $M \succeq 0$; the parameter vector $\theta = (M, A, b, \phi)$ is drawn at random with a controllable sparsity rate, and the problem dimensions (numbers of primal variables n_x and inequality constraints n_μ) are free hyperparameters. This family exercises the solver on a controlled range of KKT system sizes and, because the random instances are not guaranteed to be feasible, on a realistic mix of solvable and infeasible instances. The second family is the lane-change *trajectory game* of Section II-C (the running example), a two-player generalized Nash equilibrium problem over a planning horizon $T \in \mathbb{N}$; each sampled instance randomizes the players' initial states and lane preferences, so the batch spans a distribution of scenarios exactly as a scenario-based planner would encounter [2]. The horizon T controls the per-instance KKT dimension.

b) Baselines: The primary baseline is `PATH` [1], accessed through `PATHSolver.jl` **[TODO: cite]** and `ParametricMCPs.jl` **[TODO: cite]**; `PATH` solves one instance at a time on a single thread. We additionally report our own *unbatched* interior point solver (Algorithm 1) as an intermediate baseline: it isolates the cost of our interior-point formulation from the batching machinery, letting us separate "is the method competitive?" from "does batching help?" Against these we compare the batched interior point solver (Algorithm 2) in both its CPU (multithreaded) and GPU (cuDSS) configurations.

c) Hardware and software: All CPU experiments run on **[TODO: final CPU: model, physical core count]** with **[TODO: RAM]** of RAM; GPU experiments run on **[TODO: final GPU: model, VRAM]**. We use Julia **[TODO: version]** and report the median of **[TODO: n]** repetitions after a separate warm-up solve to exclude compilation time. **[TODO: state PATH version / license]** Unless noted, we use a convergence tolerance of $\bar{K} = 10^{-4}$; we report each batch's *solved fraction* (instances reaching $\|K_0\| \leq \bar{K}$) and its total wall-clock time to clear all B instances.

Problem	Solver	Solved	Time / solve
Random QP	`PATH`	·	·
	Algorithm 1 (UMFPACK)	·	·
	Algorithm 1 (KLU)	·	·
Trajectory game	`PATH`	·	·
	Algorithm 1 (UMFPACK)	·	·
	Algorithm 1 (KLU)	·	·

TABLE I: Single-instance reliability and speed. Solved fraction and mean per-solve wall-clock over **[TODO: n]** random instances of each family (QP: $n_x =$ **[TODO:]**, $n_\mu =$ **[TODO:]**; trajectory game: $T =$ **[TODO:]**). **[TODO: populate all cells from the fresh single-instance run; do not ship placeholders]**

B. Single-instance reliability and speed

Before batching, we evaluate the performance of Algorithm 1 vs. `PATH` `PATH` on individual problems (i.e., not batched). Table I reports, for each problem family, the fraction of instances each solver converges on and its mean **[TODO: pm standard deviation]** per-solve wall-clock. Two results are readily apparent. First, Algorithm 1 solves virtually the same fraction of problems as `PATH`, confirming Algorithm 1's reliability. Second, on a *single* instance Algorithm 1 is somewhat slower than `PATH` **[TODO: quantify the per-solve gap—expected to be a small constant factor, not an order of magnitude]**, which is unsurprising: `PATH` is a mature, heavily optimized solver, and a single solve gives our solver no opportunity to exploit computational parallelism. The batched results below show that this modest per-instance handicap is more than repaid when solving more than a handful of parametrically-related instances.

[TODO: KLU experiment: the unbatched solver currently factors ∇K_ρ with `UMFPACK` (LinearSolve.jl's default). The batched CPU path instead uses `KLU` with in-place refactorization, which reuses the pivot ordering across Newton steps and is substantially faster on these small, repeatedly-refactored systems. Re-run the single-instance baseline with `KLU` in the unbatched solver and report whether it closes (or reverses) the gap to `PATH`; update this paragraph and Table I accordingly.]

C. Batched CPU throughput vs. `PATH`

The batched solver's purpose is to clear an entire batch at once. Table II reports the total wall-clock to solve a batch of $B =$ **[TODO:]** instances of each family three ways: sequential `PATH` calls, sequential unbatched Algorithm 1 calls, and a single Algorithm 2 call across **[TODO: final thread count]** CPU threads. As shown in Figure 1(b), on the trajectory game, the CPU-multithreaded batched solver clears the batch **[TODO: 60–80]**× faster than sequential `PATH`

Problem	Solver	Total time	Solved	Speedup
Random QP	PATH	·	·	1×
	Algorithm 1	·	·	·
	Algorithm 2	·	·	·
Trajectory game	PATH	·	·	1×
	Algorithm 1	·	·	·
	Algorithm 2	·	·	·

TABLE II: Batched throughput on CPU. Total wall-clock and solved fraction to clear a batch of $B =$ **[TODO:]** instances, and speedup of the batched CPU solver over each sequential baseline. **[TODO: populate from the fresh throughput run]**

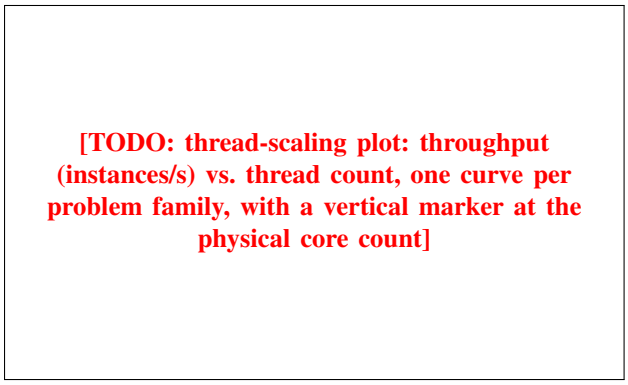


Fig. 2: Batched-solver throughput vs. CPU thread count at fixed batch size. Throughput scales with the number of active instances until the physical cores are saturated.

[TODO: finalize; confirm the QP speedup as well]. The speedup comes from two compounding sources: sharing the symbolic (G, H) structure across the batch (assembled and sparsity-analyzed once), and parallelizing the per-instance factorize/solve across threads. Note that the comparison against the sequential Algorithm 1 isolates the second effect—due purely to parallelization—from the first.

D. CPU thread scaling

Figure 2 shows batched throughput (instances cleared per second) as a function of the number of CPU threads, for a fixed batch size. Because each instance's sparse factorization is independent (cf. Section IV), throughput scales **[TODO: near-linearly? sublinearly?]** up to the number of physical cores and then **[TODO: plateaus/regresses]** when the physical cores are saturated. **[TODO: report the peak throughput and the thread count at which it is reached, for both problem families]**

E. GPU vs. CPU: a regime-dependent crossover

Depending on the setting, GPU can outperform *or* underperform multithreaded CPU operation. To characterize the tradeoffs directly, we report independent sweeps of the batch size B at fixed per-instance size (cf. Figure 3, left), and per-instance size—determined by the number of primal variables n_x in the QP setting, and by the planning horizon T in the trajectory game—at fixed B (cf. Figure 3, right).

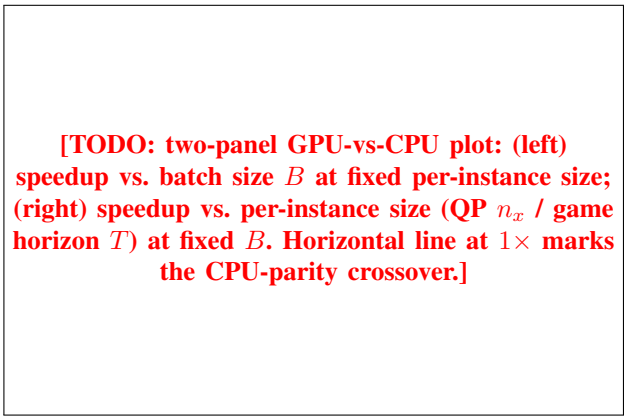


Fig. 3: GPU (`cuDSS`) vs. CPU batched throughput. The GPU overtakes the CPU only once the per-instance problem is large enough to amortize `cuDSS`'s fixed batched-factorization overhead.

The consistent pattern in Figure 3 is that the GPU backend wins once problem instances grow large enough, and is at parity or slower below that threshold. On the trajectory game, the GPU overtakes **[TODO: final thread count]**-thread CPU by **[TODO: 2.5–2.8]**× once the horizon (and thus the KKT dimension) is large enough **[TODO: state the crossover: $T \gtrsim 30$, $d \gtrsim 2000$]**; below that, and for the small dense QPs at small batch sizes, the CPU is faster or at parity **[TODO: quantify both regimes]**.

The mechanism which explains this pattern is the batched sparse factorization itself: `cuDSS` processes the whole batch as a single device-side call whose fixed launch and occupancy overhead amortizes as the size of each instance's KKT Jacobian grows, whereas the CPU's per-thread `KLU` factorization has no comparable fixed cost to amortize. **[TODO: confirm this mechanism claim against the isolated per-call factorize timing from the benchmark suite, per the "investigate, don't assume" rule—the crossover should be visible in raw `jacobian!+factorize!` timing, not just end-to-end.]**

Two caveats bound this result and are reported plainly. First, the fraction of trajectory games solved in each batch declines at longer time horizons **[TODO: state the numbers, e.g. ∼X% at $T = 50$]**, so the large-T GPU-vs-CPU ratios compare backends on a harder instance distribution, not just a larger one. Second, the QP problem-size sweep is confounded by the fraction of solved random instances changing sharply with the primal dimension n_x. **[TODO: after the fresh run, restate these caveats with final numbers, and note the large-batch (`cuDSS`) OOM behavior if it still reproduces.]**

VI. Discussion and Future Work

[TODO: TODO]

References

[1] S. P. Dirkse and M. C. Ferris, "The path solver: a nommonotone stabilization scheme for mixed complementarity problems," *Optimization methods and software*, vol. 5, no. 2, pp. 123–156, 1995.

[2] J. Li, C. Y. Chiu, L. Peters, F. Palafox, M. Karabag, J. Alonso-Mora, S. Sojoudi, C. Tomlin, and D. Fridovich-Keil, "Scenario-game admm: A parallelized scenario-based solver for stochastic noncooperative games," in *62nd IEEE Conference on Decision and Control, CDC 2023*. IEEE, 2023, pp. 8093–8099.

[3] L. Peters, A. Bajcsy, C.-Y. Chiu, D. Fridovich-Keil, F. Laine, L. Ferranti, and J. Alonso-Mora, "Contingency games for multi-agent interaction," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2208–2215, 2024.

[4] B. Amos and J. Z. Kolter, "Optnet: differentiable optimization as a layer in neural networks," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, 2017, pp. 136–145.

[5] F. Facchinei and J.-S. Pang, *Finite-dimensional variational inequalities and complementarity problems*. Springer, 2003.